PUB AttendEye – Project Documentation

Oct 28, 2026 · @Feroz

1. Overview

PUB AttendEye (v2.0.0) is an AI-powered attendance and academic management platform for Pundra University of Science & Technology (PUB), Bogra, Bangladesh. It replaces roll calls and paper sign-in sheets with face-recognition attendance, role-based portals for students, teachers and administrators, and reliable PostgreSQL records for every course.

The project was built for the **PUB CSE Fest 2026 Hackathon**, Track B (Artificial Intelligence and Machine Learning), problem **B4: Computer-Vision-Based Smart Attendance System**. Version 2.0 is a full architectural overhaul of the legacy v1 prototype, moving to a Next.js App Router frontend, Supabase PostgreSQL with SSR authentication, and a Python FastAPI vision engine.

Item	Detail
Product	PUB AttendEye, version 2.0.0
Institution	Pundra University of Science & Technology, Bogra, Bangladesh
Event	PUB CSE Fest 2026 Hackathon, Track B, problem B4
Web application	Next.js (App Router), TypeScript, Tailwind CSS, port 3000
Backend and auth	Supabase PostgreSQL with Row Level Security and SSR Auth
Vision engine	Python FastAPI with OpenCV DNN, YuNet face detection, SFace 128-dimension embeddings, port 8000
Repository	GitHub: feroz-afm / Computer-Vision-Based-Smart-Attendance-System-for-Pundra-University

2. Problem statement

Calling the roll wastes lecture time, and paper sign-in sheets let students mark attendance for absent friends (proxy attendance). Universities also struggle to keep consistent, auditable attendance records per course, and students rarely learn in time that their attendance is falling below the limit.

The goal is a system that takes attendance from a live camera feed, recognises enrolled students in a single frame, sends uncertain or unknown faces to the teacher instead of guessing, logs every manual correction, and gives students and administrators course-wise attendance reports.

Track B: Artificial Intelligence and Machine Learning

B4. PUB AttendEye

Computer-Vision-Based Smart Attendance System

Problem Description

Calling the roll wastes class time and paper sign-in sheets allow proxy attendance. Build a system that takes attendance from a live camera feed or a photo of the class, recognises enrolled students, lets the teacher confirm uncertain cases, and keeps reliable attendance records for every course.

Required Features

1. Student enrolment with consent, capturing several face samples per student and linking them to student ID, course and section.
2. The teacher starts a class session (course, section, time window) and captures attendance from a live camera feed or an uploaded classroom photo.
3. Detect and recognise multiple faces in a single frame and mark matched students present.
4. Confidence threshold: unknown or low-confidence faces are sent to the teacher for review instead of being marked automatically.
5. Liveness or anti-spoofing check against a printed photo or a face shown on a phone screen.
6. Manual correction by the teacher; every correction is logged with time and reason.
7. Students see their attendance percentage for each course and receive an alert when it falls below a configurable limit.
8. Course-wise and date-wise reports with CSV or PDF export.
9. Evaluation on a small test set under different lighting and angles: report precision and recall (or false-accept and false-reject rates) and justify the chosen threshold.

Users and Roles

Student	Enrols face samples with consent and views own attendance.
Teacher	Runs sessions, reviews flagged faces, corrects records, exports reports.
Administrator	Manages courses, sections, users and the attendance limit.

Restrictions

- Use only faces of consenting volunteers (for example, team members). Do not use scraped images of real students.
- Store face embeddings rather than raw photos where possible, and protect all biometric data.
- Pretrained detection and recognition models may be used, but the team must explain the complete pipeline.

Optional Features

Handling masks and partial occlusion, QR plus face two-factor check-in, CCTV or IP-camera stream, offline mobile mode, attendance analytics dashboard.

Be creative: the features listed above are the minimum scope. Teams are encouraged to add new features or modify the listed ones wherever it makes the solution more useful. Well-justified additions are credited under Innovation.

Figure 1. The official problem set for Track B, problem B4.

Constraints from the problem set

- Use only faces of consenting volunteers, never scraped images.

- Store face embeddings rather than raw photos, and protect all biometric data.
- Pretrained detection and recognition models are allowed, but the full pipeline must be explained.

3. Methodology

The attendance pipeline runs in eight stages. Each stage is kept separate so its threshold or model can be tuned without touching the rest.

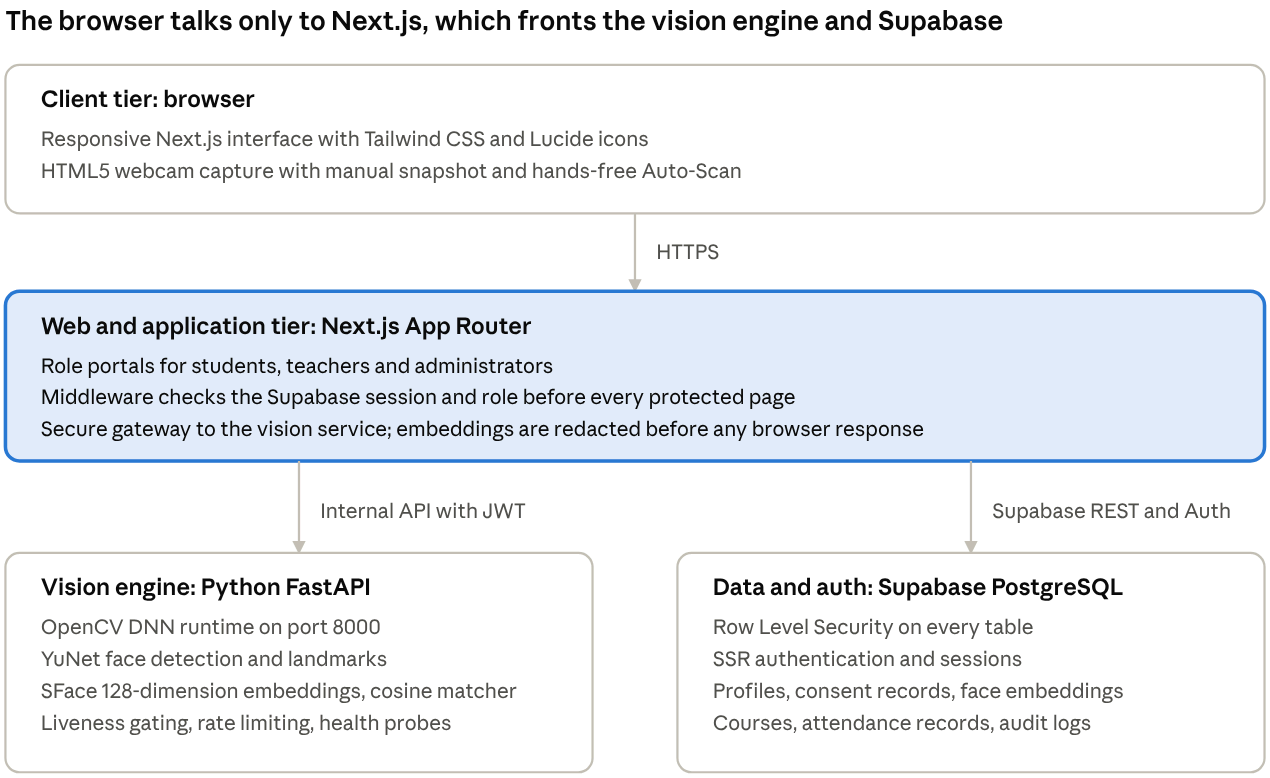
1. **Consent and enrolment.** The student gives voluntary consent, then captures face samples from the webcam. Consent is stored with a time stamp.
2. **Face detection.** The YuNet detector finds every face in a frame and returns bounding boxes and facial landmarks.
3. **Alignment and embedding.** Landmarks are used to align each face crop. The SFace model converts it into a 128-dimension vector, normalised to unit length. Only this vector is stored.
4. **Liveness gating.** Geometry, landmark and sharpness checks reject flat or blurry inputs such as a printed photo or a phone screen. An active head-turn or blink challenge is planned for v2.1.
5. **Roster-scoped matching.** Each live face is compared by cosine similarity only against students enrolled in the selected course and section. The best score and its margin over the runner-up are recorded. A collision rule stops one student ID being claimed by two faces in the same frame.
6. **Threshold decision.** A score of 0.65 or more is marked Present. A score from 0.55 to 0.65, or an ambiguous margin, goes to the teacher's Review Queue. Below 0.55 the face is Unknown and is not marked.
7. **Persistence and correction.** Results are written to PostgreSQL through Supabase with time stamp and confidence. Teachers can correct any record, and every correction needs a written reason that goes to the audit trail.
8. **Reporting and alerts.** Attendance percentage is computed per course from recorded sessions. Students see a notice below 75% and a critical warning below 60%, and teachers export Excel or PDF sheets.

Why these design choices

Choice	Reason
Embeddings instead of photos	A leaked database exposes no usable face images, which meets the privacy restriction
Roster-scoped search	Comparing against 30 to 40 enrolled students instead of the whole university lowers false matches
Two thresholds plus a margin	Confident matches are automatic, doubtful ones get a human decision, so a wrong match is rarely silent
Pretrained YuNet and SFace	Lightweight, open-licence (Apache 2.0) models that run on a normal CPU without extra training data

4. System architecture

AttendEye is a three-tier system. The browser never calls the vision engine or the database for protected data directly: every request passes through the Next.js tier, which validates the session and role first.

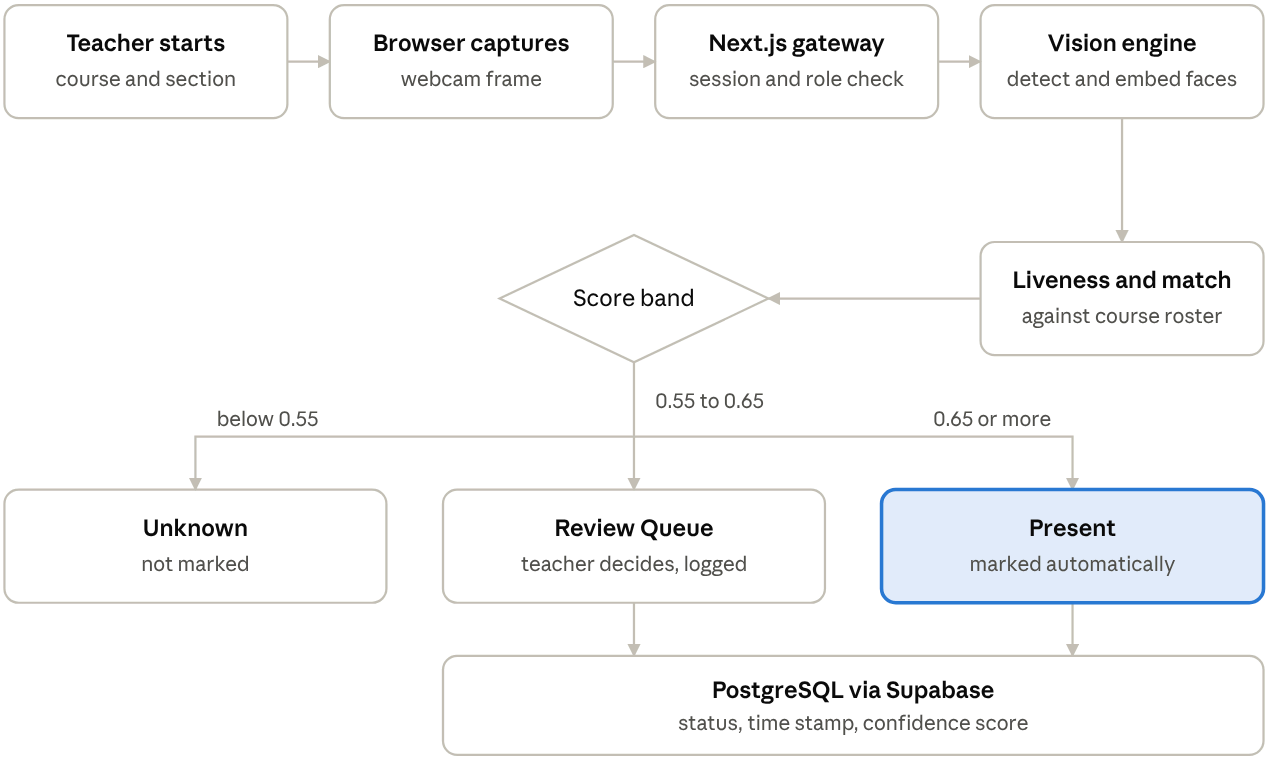


system architecture · 3 tiers, 4 components

The web tier sends captured frames to the Python service over an internal, token-checked API and reads or writes records in Supabase. Only match results, not embeddings, travel back to the browser.

Data flow of one scan.

One scan: the score decides whether a face is marked, reviewed or ignored



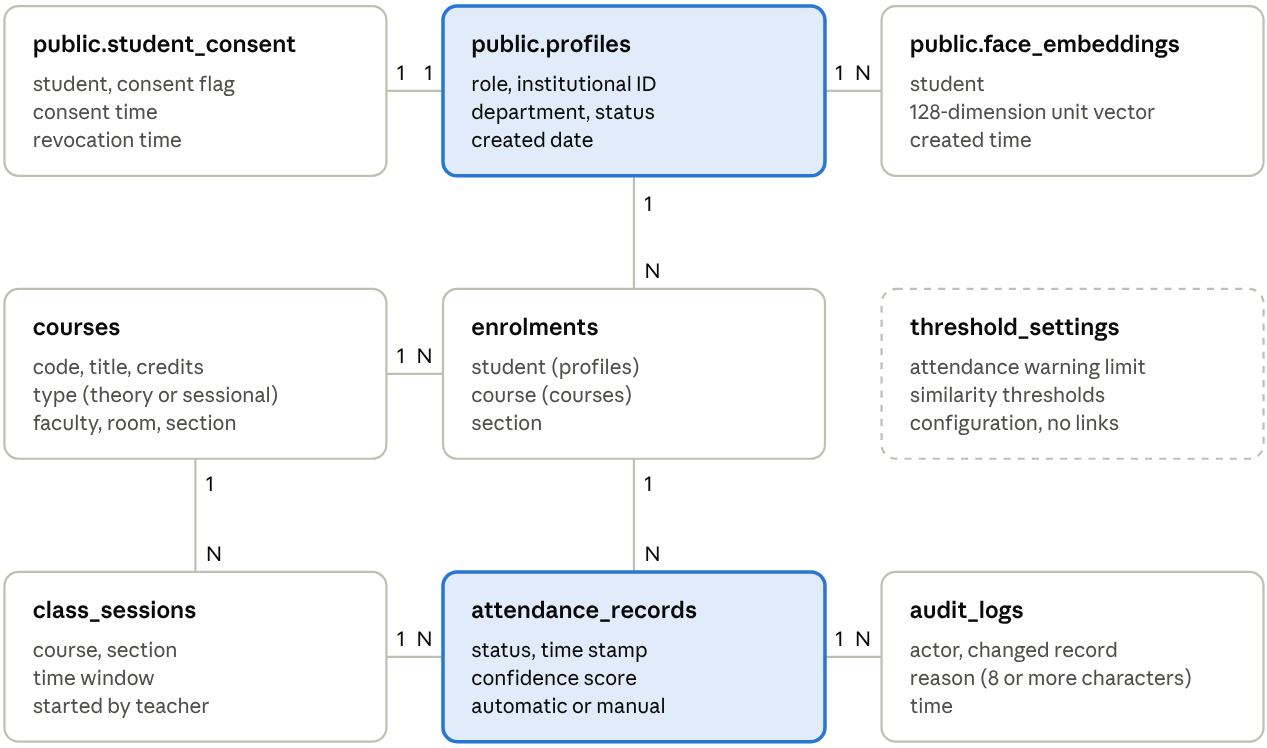
attendance data flow · one scan, 3 outcomes

Only Present and Review Queue results are written to the database. Unknown faces are not marked, so a stranger in the frame never creates a record.

5. Database design

All data lives in Supabase PostgreSQL. Row Level Security policies restrict each role to the rows it needs, and the schema is created by the baseline SQL file plus three Version 2 migrations (see section 10).

Every attendance record ties a student, a course session and an audit trail



1 = one row, N = many rows. Names with the public. prefix are confirmed in the README.
The other entities are logical, shown as the application behaves. Dashed box = configuration only.

entity relationship diagram · 9 entities

A profile owns its consent and embeddings. A student reaches attendance through an enrolment in a course, and each attendance record belongs to one class session and can collect audit log entries.

Entity	Purpose	Key fields
`public.profiles`	One row per user: identity, role and approval state	Role (student, teacher, admin), institutional ID, department, status (for example `pending_approval`, active), created date
`public.face_embeddings`	Unit-normalised 128-dimension face vectors per student, never raw photos	Student, vector, created time

Entity	Purpose	Key fields
`public.student_consent`	Biometric consent record	Student, consent flag, consent time, revocation time
Course catalog	Curriculum courses for CSE 3rd year, 1st semester	Course code, title, credits, type (Theory or Sessional), assigned faculty, room, section
Enrolments	Links a student to a course and section, and defines the recognition roster	Student, course, section
Class sessions and attendance records	One session per class, one record per student	Status (Present, Late, Absent), time stamp, confidence score, source (automatic or manual)
Review queue	Low-confidence or ambiguous matches waiting for the teacher	Session, candidate student, score, decision
Audit logs	Every manual override	Actor, record changed, reason (at least 8 characters), time
Threshold settings	Attendance limit and recognition thresholds, edited in the admin Threshold Settings tab	Attendance warning percentage, similarity thresholds

The three `public.` table names are taken from the project README. The other entities are described as the application shows them, so check exact table and column names against the files in `supabase/migrations` before submission.

Relationships

- A profile has zero or many face embeddings and at most one active consent record.
- A course has many enrolments, and each enrolment links one student profile to it.
- A class session belongs to one course and section, and has many attendance records.
- An attendance record may have many audit log entries, one for each manual correction.

- A teacher profile can teach many courses, but only after an administrator approves the account.

6. Key features

Role isolation and protected portals. Dedicated login gateways serve students, teachers and administrators. Server-side Next.js middleware checks session tokens against the `public.profiles` table in PostgreSQL, so a user cannot open another role's dashboard. Navigation shows only the active session's actions, and a logged-in user can sign out deliberately from any portal.

Faculty registration and admin approval. Teachers self-register with their institutional email. New accounts start in a `pending_approval` state that blocks attendance sessions until an administrator approves them in the review queue, where the admin can also assign courses.

Ethical biometric consent.

- Raw photographs are never stored permanently in the database.
- Only unit-normalized 128-dimensional embeddings are kept, in `public.face_embeddings`, and they are never returned to client-side JavaScript.
- A student must give voluntary consent, recorded in `public.student_consent` with a revocation timestamp, before enrolment.

Live optical classroom attendance.

- Real-time webcam recognition with hands-free Auto-Scan mode (a 4-second interval).
- Multi-face detection, so several students are matched in one frame.
- Roster-scoped matching: candidates are limited to students enrolled in the selected course and section.
- Multi-face collision protection prevents the same ID from being claimed twice in one frame.

Tiered decision thresholds. Each match is classified by cosine similarity and the margin to the runner-up.

Outcome	Rule	Action
Present	Similarity at or above 0.65	Marked present automatically

Outcome	Rule	Action
Review Queue	Similarity between 0.55 and 0.65, or ambiguous margin	Sent to the teacher for review
Unknown	Similarity below 0.55	Classified as unknown, not marked

Manual adjustments and audit trail. Teachers can correct any record. The dialog requires a justification of at least 8 characters, which is logged to the database audit trail with time and reason.

Student performance and warning indicators. Attendance percentage is calculated live from recorded class sessions. A notice appears below 75% and a critical warning below 60%.

One-click Windows launchers. `start-attendeye.bat` validates the environment, models and ports, then starts the backend and frontend. `stop-attendeye.bat` cleanly ends background processes on ports 8000 and 3000.

7. Hackathon requirement coverage

The B4 problem set lists nine required features. This table maps each one to the current implementation. Anything the project material does not confirm is marked as such.

#	Required feature	How AttendEye covers it	Status
1	Student enrolment with consent, several face samples, linked to ID, course and section	Consent gate (`public.student_consent`), embeddings stored per student, enrolment tied to student ID and course roster	Built
2	Teacher starts a session (course, section, time window) and captures from live camera or photo	Teacher portal one-click session launch with webcam capture	Built (live camera); photo upload not confirmed
3	Detect and recognise multiple faces in one frame	YuNet detection plus SFace embeddings, multi-face frames with collision protection	Built

#	Required feature	How AttendEye covers it	Status
4	Confidence threshold: unknown or low-confidence faces go to teacher review	0.65 auto-present, 0.55 to 0.65 or low margin to the Review Queue, below 0.55 unknown	Built
5	Liveness or anti-spoofing against printed photo or phone screen	Geometry, landmark and sharpness gating in this release; active head-turn or blink challenge planned for v2.1	Partial
6	Manual correction by teacher, logged with time and reason	Correction dialog requires at least 8 characters of justification, written to the audit trail	Built
7	Students see attendance percentage per course and get an alert below the limit	Live percentage per course; notice below 75%, critical warning below 60%	Built
8	Course-wise and date-wise reports with CSV or PDF export	Teacher portal exports Excel and PDF sheets; CSV export of roster and catalog	Built
9	Evaluation under different lighting and angles: precision, recall, justified threshold	Python biometric pipeline tests exist; measured precision and recall figures are not in the project material	To document

Roles. Students enrol face samples with consent and view their own attendance. Teachers run sessions, review flagged faces, correct records and export reports. Administrators manage courses, sections, users and the attendance limit (the Threshold Settings tab).

Restrictions followed. Only embeddings are stored, not raw photos; pretrained models (OpenCV Zoo YuNet and SFace) are used and the full pipeline is explained in this document.

8. Portals and system snapshot

The web app (`localhost:3000`) is a dark-themed Next.js interface with a shared header (Overview, Portals, How It Works, Team, University) and a live-backend banner that confirms the connection to Supabase PostgreSQL and Authentication.

Landing page

The home page presents the product as "Smart Attendance. Automated with AI.", with a Vision Node preview card showing the biometric architecture, plus feature highlights for liveness and anti-proxy checks, instant verification and automated reports. Below it are the role portals, a four-step "How It Works" pipeline, the hackathon team section and an institutional context section for Pundra University.

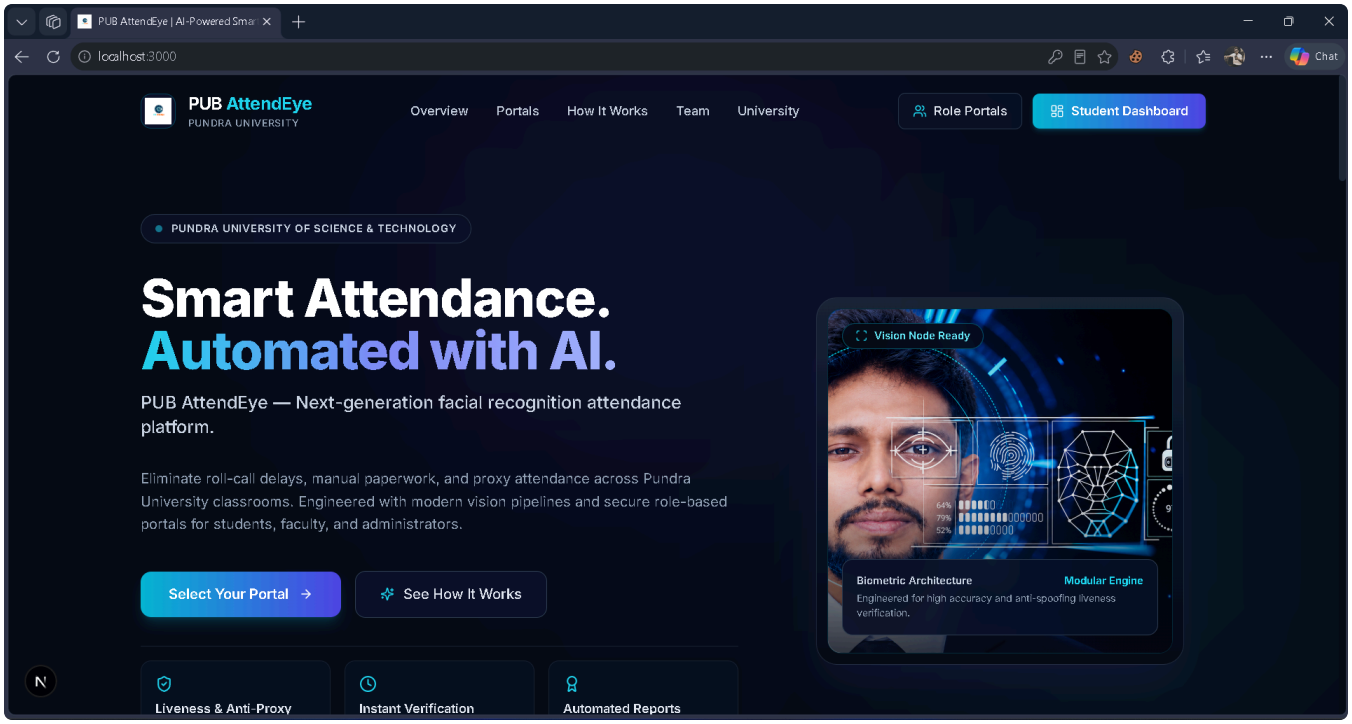


Figure 2. Landing page: product pitch, Vision Node preview and entry buttons.

How it works, in four steps

1. **Camera feed initiation.** The instructor starts a verified class session from their portal, capturing the lecture hall.
2. **Facial detection and landmark alignment.** The vision pipeline extracts face crops and computes biometric embeddings with anti-spoofing verification.
3. **PostgreSQL auto-record via Supabase.** Attendance status (Present, Late, Absent) is matched and committed instantly with timestamp and confidence metrics.

4. **Dynamic analytics and reports.** Students receive instant verification, and instructors and department chairs get filterable attendance summaries.

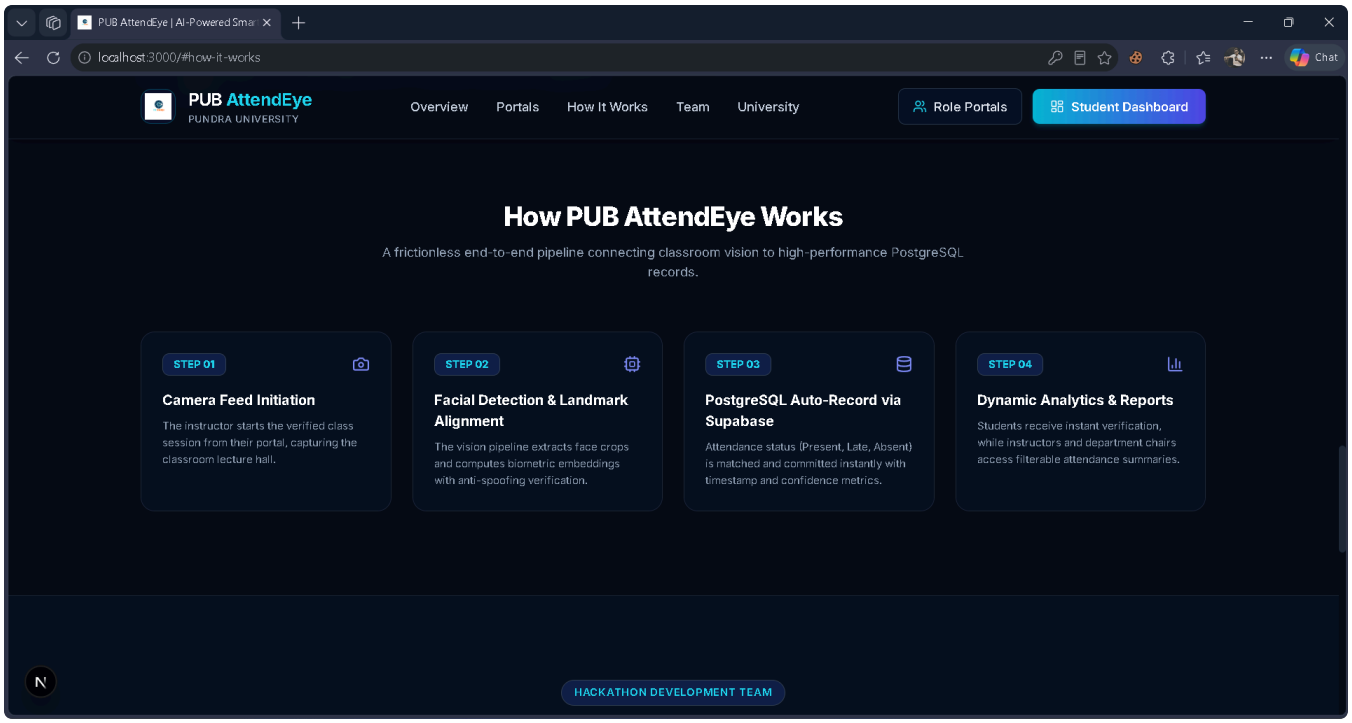


Figure 3. The four-step pipeline as shown on the landing page.

Role portals

Portal	Audience	Core capabilities
Student Portal	Undergraduate and graduate students	Real-time attendance history per course; eligibility warning for exam requirements; biometric profile verification status
Teacher Portal	Faculty and instructors	One-click automated class session launch; live student face detection validation; export of Excel and PDF attendance reports
Administrator Portal	University management	Centralized student and teacher management; course catalog and semester enrolments; system-wide attendance audit logs

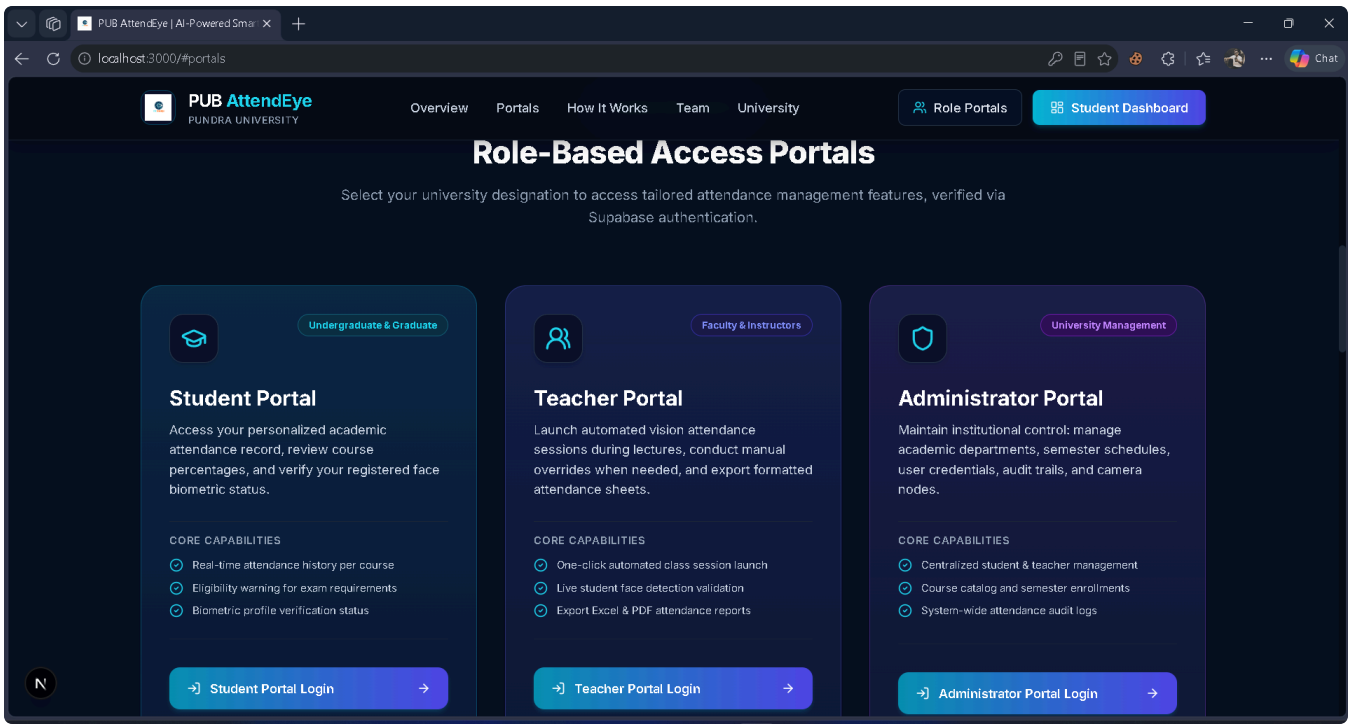


Figure 4. Role-based access portals with login buttons for each role.

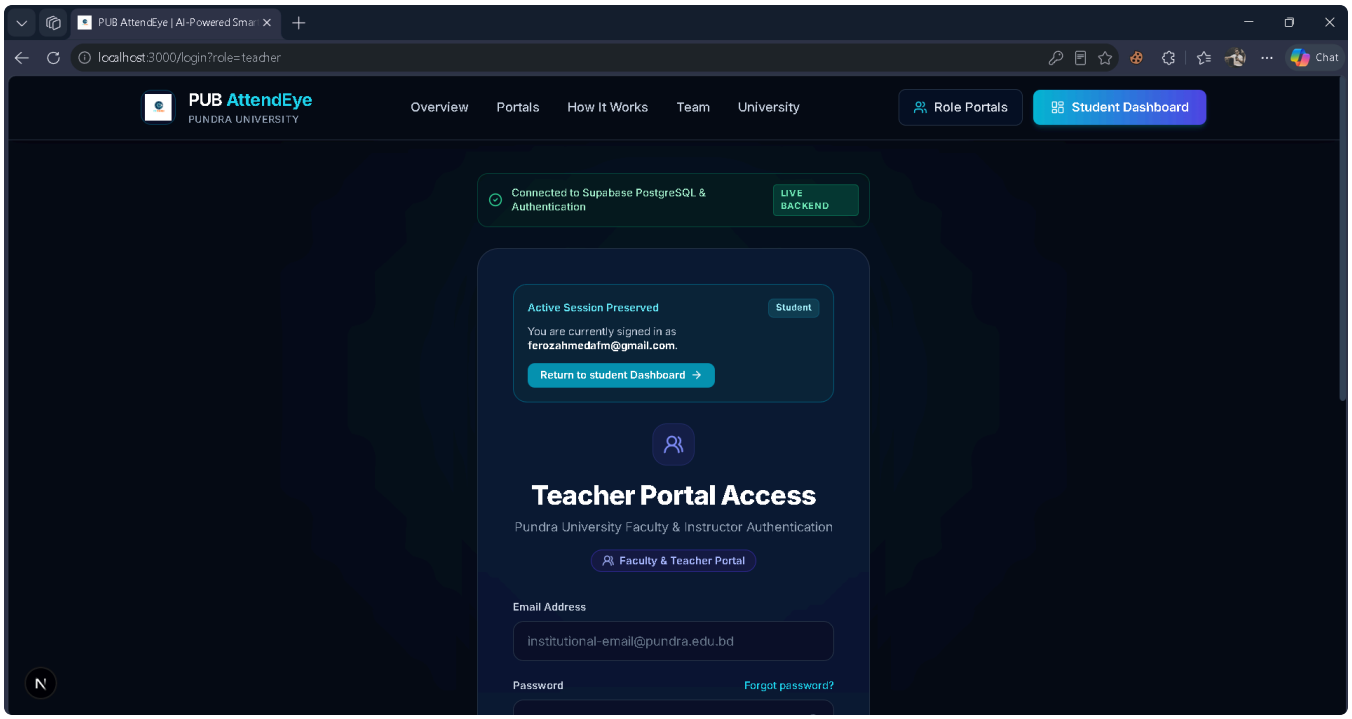


Figure 5. Teacher portal access page. The "Active Session Preserved" card shows an existing signed-in session and a shortcut back to its dashboard.

Student dashboard

The student dashboard shows four status cards: overall attendance (with classes held and attended), academic status, face recognition consent (Pending Consent until the student provides it) and registered courses with credit totals. Below them, a course-wise attendance list covers the current semester (5th Semester, B.Sc. in Computer Science & Engineering). Actions include Change Photo, Export Roster CSV, Home and Sign Out. When a student has no enrolments, the page shows a clear "No Enrolled Courses Found" state that points them to their department academic coordinator.

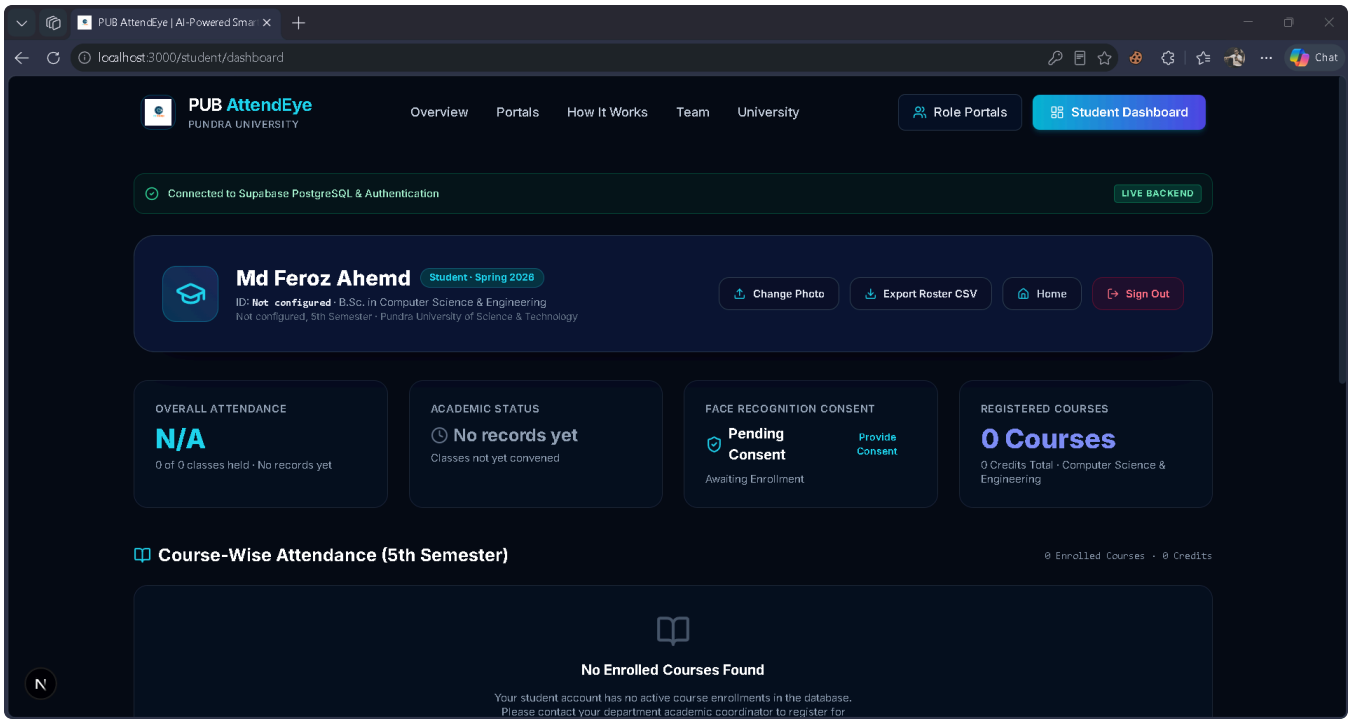


Figure 6. Student dashboard in its empty state: no enrolments, consent pending.

Administrator console

The System Administrator Console (Superuser Level) is organised into five tabs: Curriculum Courses, Teacher Approvals, User Directory, Audit Logs and Threshold Settings. The summary cards currently read:

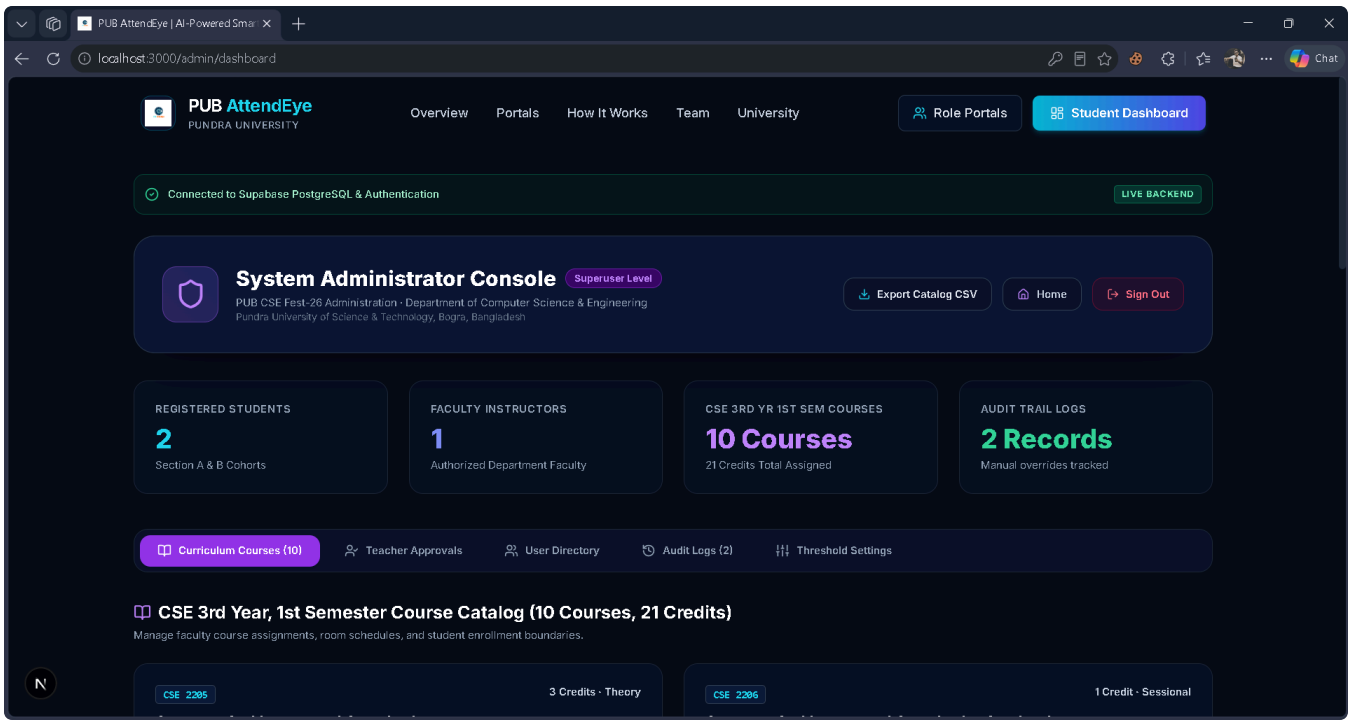


Figure 7. Administrator console: summary cards and the five management tabs.

Metric	Value	Note
Registered students	2	Section A and B cohorts
Faculty instructors	1	Authorized department faculty
CSE 3rd year 1st semester courses	10	21 credits total assigned
Audit trail logs	2	Manual overrides tracked

Course catalog (CSE 3rd Year, 1st Semester). Each course card shows credits and type, the assigned faculty, room and section, the enrolled-student count and a Reassign Faculty action. The catalog can be exported to CSV. Courses visible in the current build:

Code	Course	Credits	Type	Room	Enrolled
CSE 2205	Computer Architecture and Organization	3	Theory	Room 402, Section A	32
CSE 2206	Computer Architecture and Organization Sessional	1	Sessional	Hardware Lab 2, Section A	32
CSE 3103	Operating Systems	3	Theory	Room 403, Section A	28

Code	Course	Credits	Type	Room	Enrolled
CSE 3104	Operating Systems Sessional	1	Sessional	Software Lab 1, Section A	28
CSE 2201	Web Engineering	3	Theory	Not shown	Not shown
CSE 2202	Web Engineering Sessional	1	Sessional	Not shown	Not shown
LAW 3201	Cyber and Intellectual Property Law	2	Theory	Auditorium B, Section A	34
BUS 3101	Technology Entrepreneurship for Business	3	Theory	Seminar Hall, Section A	34

The catalog reports 10 courses in total; two further courses were not visible in the captured screens.

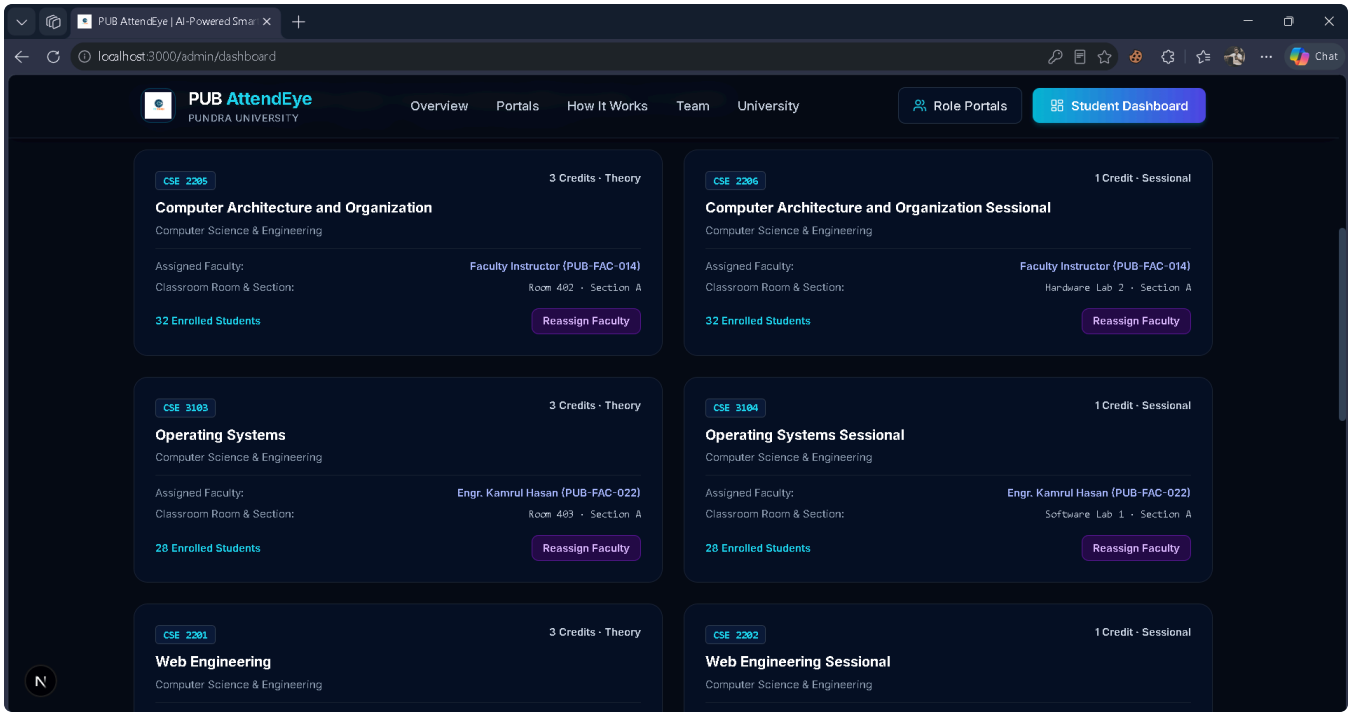


Figure 8. Course catalog cards with faculty, room, section, enrolled count and Reassign Faculty.

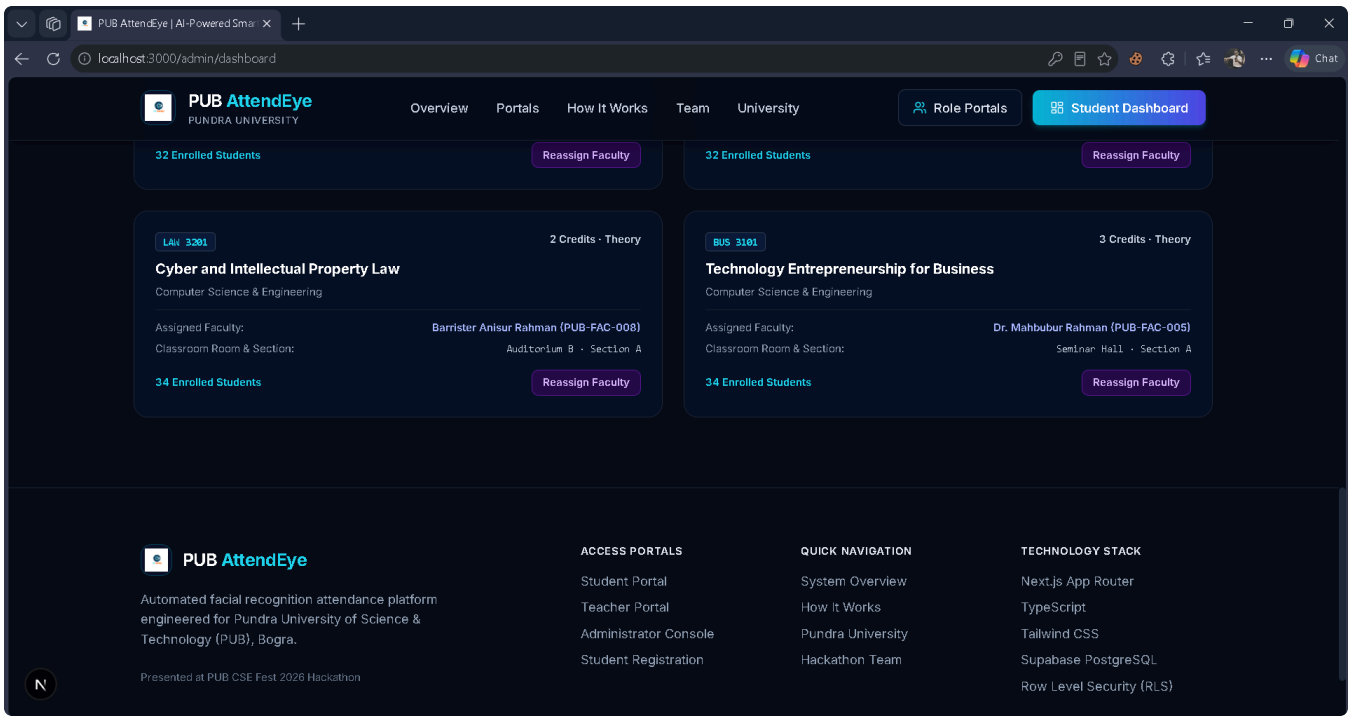


Figure 9. The end of the catalog and the site footer listing portals and the technology stack.

User directory and role permissions. The directory lists each user with institutional ID, department, role (Teacher, Student or Admin), status and creation date, with a search box and role filters (All, Student, Teacher, Admin). The current seed data holds four active accounts, all in Computer Science & Engineering: one teacher, two students (including a diagnostic test student) and one administrator, created between 22 and 27 October 2026.

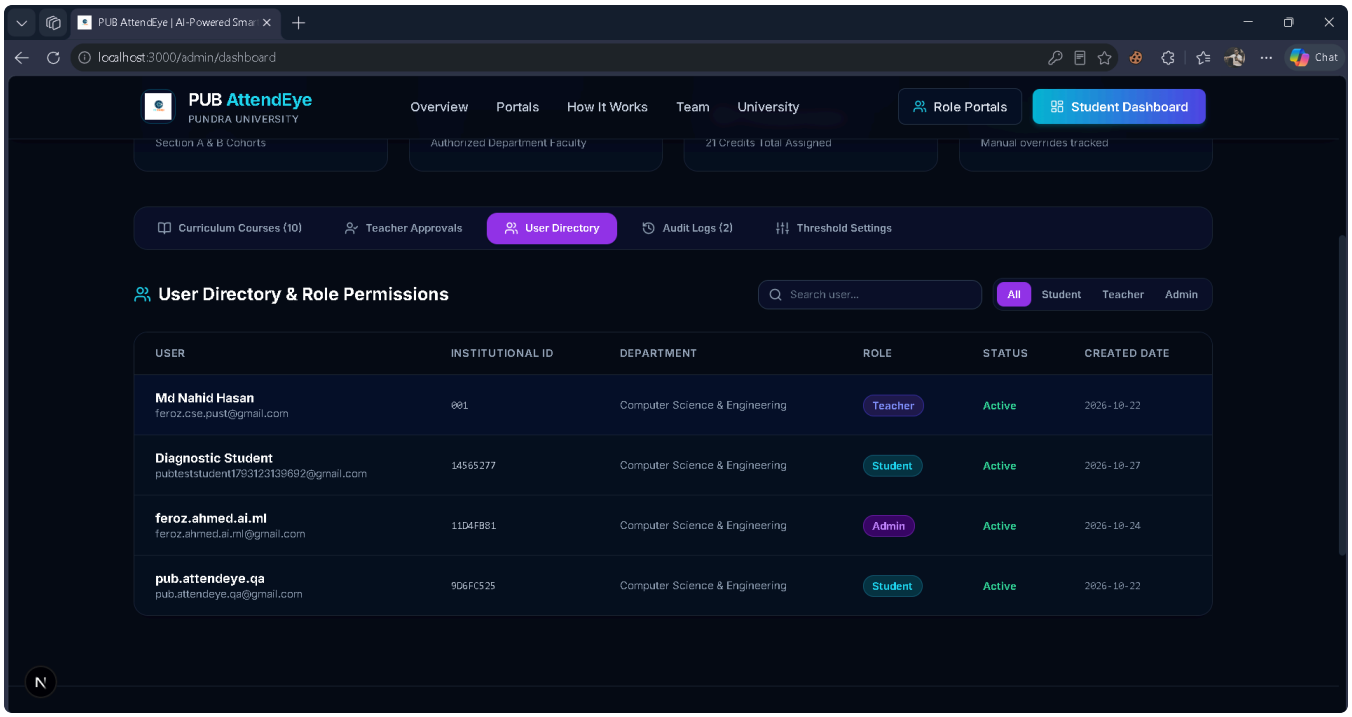


Figure 10. User directory with role filter, search, status and creation date.

9. Technology stack and prerequisites

Layer	Technology
Web framework	Next.js App Router (React, server components, route handlers, middleware)
Language and styling	TypeScript, Tailwind CSS, Lucide icons, responsive HTML5 camera capture
Database and auth	Supabase PostgreSQL with Row Level Security and SSR Auth
Vision service	Python FastAPI served by Uvicorn
Face detection	YuNet via OpenCV DNN (OpenCV Zoo, Apache 2.0)
Face recognition	SFace 128-dimension embeddings (OpenCV Zoo, Apache 2.0)
Matching	Cosine similarity with a runner-up margin check

Prerequisites

- Node.js v18.17.0 or v20+ (LTS recommended)
- Python v3.10 or v3.11 (v3.11 recommended for OpenCV DNN)
- A Supabase project (cloud-hosted or local CLI)
- A webcam or USB camera for optical capture

10. Setup and running

10.1 Environment variables

Copy the example file to `.env.local` in the project root.

```
# Windows PowerShell
Copy-Item .env.example .env.local

# Linux / macOS
cp .env.example .env.local
```

Variable	Required	Default or description
`NEXT_PUBLIC_SUPABASE_URL`	Yes	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Yes	Supabase public anon key (safe for the browser client)
`SUPABASE_SERVICE_ROLE_KEY`	Optional	Backend-only admin key, never exposed to the browser
`NEXT_PUBLIC_APP_NAME`	No	"PUB AttendEye"
`NEXT_PUBLIC_UNIVERSITY_NAME`	No	"Pundra University of Science & Technology"
`PYTHON_CV_API_URL`	Yes	URL of the Python backend, for example `http://127.0.0.1:8000`
`SUPABASE_JWT_SECRET`	Optional	Project JWT secret for legacy HS256 verification
`SUPABASE_JWT_JWKS_URL`	Optional	Custom JWKS endpoint for asymmetric ES256/RS256 JWT checks
`PYTHON_API_ENV`	No	"development" (allows a local dev token) or "production"
`FRONTEND_ORIGIN`	No	Allowed CORS origin, for example `http://localhost:3000`

> Never commit `.env.local`, `.env`, service role keys or production JWT secrets to version control.

10.2 Database and Supabase schema

1. Open your Supabase Dashboard, then SQL Editor.
2. Run the baseline schema file from the `supabase` folder.

3. Run the three Version 2 migrations in order, from the `supabase/migrations` folder.

To create the first administrator, promote an existing account to the `admin` role in the SQL Editor:

```
SELECT public.promote_to_admin('admin@pundra.edu.bd');
```

10.3 Option A: one-click launcher (Windows, recommended)

Double-click `start-attendeye.bat`. It validates the environment, models and ports, then starts both services. To shut everything down cleanly, run `stop-attendeye.bat`.

10.4 Option B: manual startup

Python vision service (port 8000)

```
# 1. Create and activate a virtual environment
python -m venv .venv
.venv\Scripts\activate        # Windows
source .venv/bin/activate     # Linux / macOS

# 2. Install dependencies
pip install -r requirements.txt

# 3. Make sure the YuNet and SFace ONNX model files are in the models folder

# 4. Launch FastAPI
uvicorn backend.api:app --host 127.0.0.1 --port 8000
```

If your entry module has a different name, adjust the `backend.api:app` part. Verify the service responds on `http://127.0.0.1:8000` before starting the web app.

Next.js web application (port 3000)

```
npm install
npm run dev            # development
# or
npm run build && npm start   # production
```

Open the application at `http://localhost:3000`.

11. Verification and testing suite

The repository ships automated tests covering TypeScript compilation, the production bundle, the Python vision pipeline and end-to-end role flows.

1. Static type and production build checks

```
npx tsc --noEmit    # TypeScript type safety
npm run build       # optimised production build
```

2. Python unit and biometric pipeline tests

```
python -m unittest discover -s tests -p "test_*.py"
```

3. End-to-end system and regression tests (Node scripts in the `tests` folder)

Area	What is verified
Session and teacher isolation	A session or role cannot leak into another portal
Biometric enrolment, consent and RLS	Consent is required, embeddings are stored securely, row-level security holds
Classroom attendance readiness	Course and section roster matching and multi-face detection logic
Teacher registration	Institutional email support and sign-up flow
Faculty approval workflow	Pending, approved and rejected states, with administrator actions
Student dashboard sync	Dashboard data matches the real database records

Evaluation under different lighting and angles. Requirement 9 of the problem set asks for precision and recall (or false-accept and false-reject rates) on a small test set, and a justified threshold. The thresholds used are 0.65 for automatic present and 0.55 for review. Record your measured results here before submission.

Condition	Test faces	True accepts	False accepts	False rejects	Precision	Recall
Normal indoor light, frontal						
Low light						

Condition	Test faces	True accepts	False accepts	False rejects	Precision	Recall
Side angle (about 45 degrees)						
Printed photo or phone screen (spoof)						

12. Security and privacy invariants

- **No raw biometric storage.** The system never saves photographic face samples to disk or database tables.
- **Vector redaction.** The 128-dimension embeddings are never transmitted to client-side browser code; they stay inside the trusted server and vision bridge.
- **Token isolation.** JWT validation is established directly through Supabase JWT verification, not a trusted client claim.
- **Anti-collusion controls.** Faculty cannot self-approve, and cannot assign courses without administrator authorization.
- **Row Level Security.** Supabase RLS policies protect profiles, enrolment and attendance tables per role.
- **Consent first.** Enrolment is blocked until the student gives voluntary consent, which can be revoked and is time-stamped.

13. Limitations and future work

Area	Current behaviour	Guidance or plan
Camera permissions (HTTPS)	Modern browsers restrict `navigator.mediaDevices.getUserMedia`. `localhost` is exempt, but production deployments need SSL/TLS (HTTPS)	Serve production over HTTPS
Classroom lighting and angles	Accuracy drops under poor lighting; extreme side profiles (above about 45 degrees) or strong backlighting may be routed to the Review Queue	Use even front lighting; teachers confirm flagged faces

Area	Current behaviour	Guidance or plan
Anti-spoofing	This release uses geometry, landmark checks and sharpness gating	Active 3D challenge-response (head turn or blink) scheduled for v2.1
Resolution for large lectures	Large halls (40 or more students per frame) need a higher-resolution capture	Use 1080p Full HD or split the room into segmented scans
Role routing (observed)	In the captured screens, a teacher-role account opened `/student/dashboard` and saw placeholder values ("Not configured"), and the header still showed the student dashboard button on an administrator page	Confirm the middleware redirects each role to its own portal and that header links follow the signed-in role
Empty data state	The student dashboard shows 0 courses and N/A attendance until an administrator enrols the student	Enrol students in the course catalog before demo

Optional features from the problem set not yet covered: mask and partial-occlusion handling, QR plus face two-factor check-in, CCTV or IP-camera streams, and offline mobile mode.

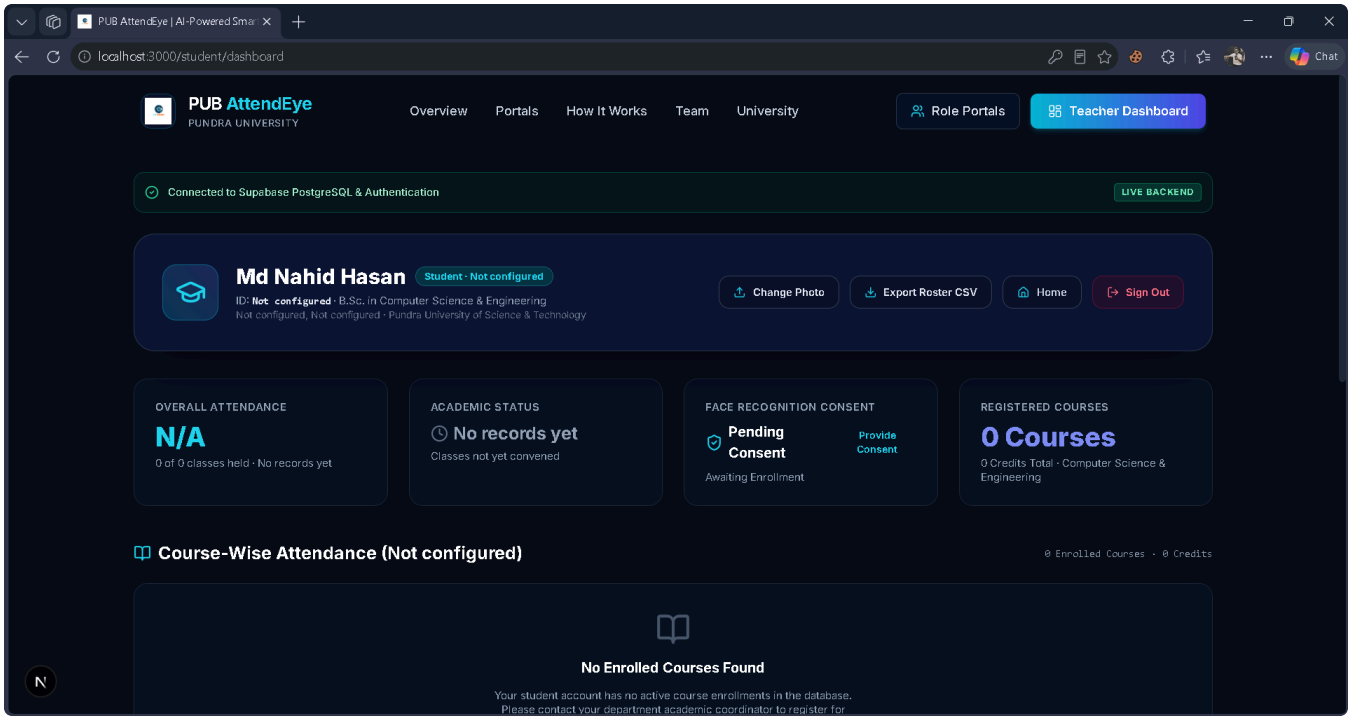


Figure 11. Observed behaviour: the header offers a Teacher Dashboard button, while the page at /student/dashboard renders a student view with "Not configured" values.

14. Team and credits

PUB AttendEye was developed by **Team AttendEye** for Pundra University of Science & Technology (PUB), Bogra, as a Computer Science & Engineering project for the PUB CSE Fest 2026 Hackathon. The team page credits the following responsibilities:

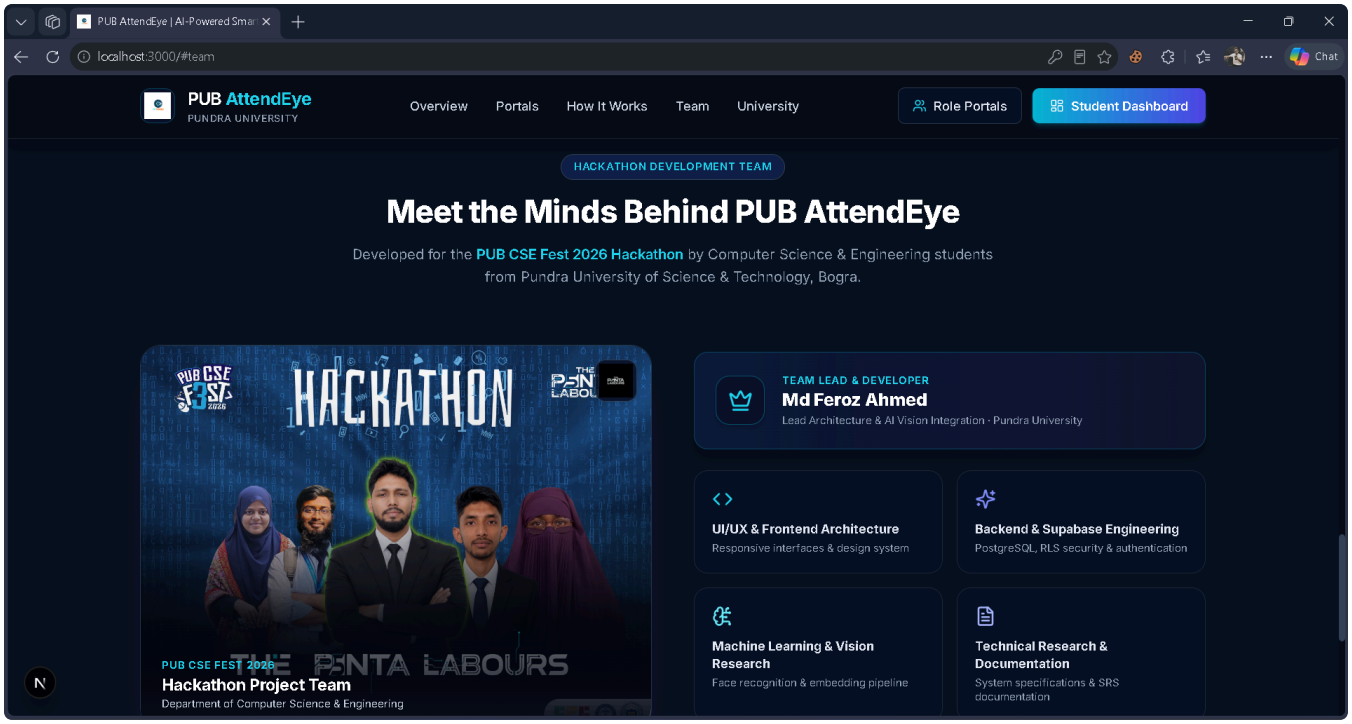


Figure 12. Team section of the landing page, presented at PUB CSE Fest 2026.

Role	Focus
Team lead and developer: Md Feroz Ahmed	Lead architecture and AI vision integration
UI/UX and frontend architecture	Responsive interfaces and design system
Backend and Supabase engineering	PostgreSQL, RLS security and authentication
Machine learning and vision research	Face recognition and embedding pipeline
Technical research and documentation	System specifications and SRS documentation

Open-source models and licences

- **YuNet face detector:** OpenCV Zoo (Apache License 2.0)

- **SFace face recognizer:** OpenCV Zoo (Apache License 2.0)

The project is released as open-source software.

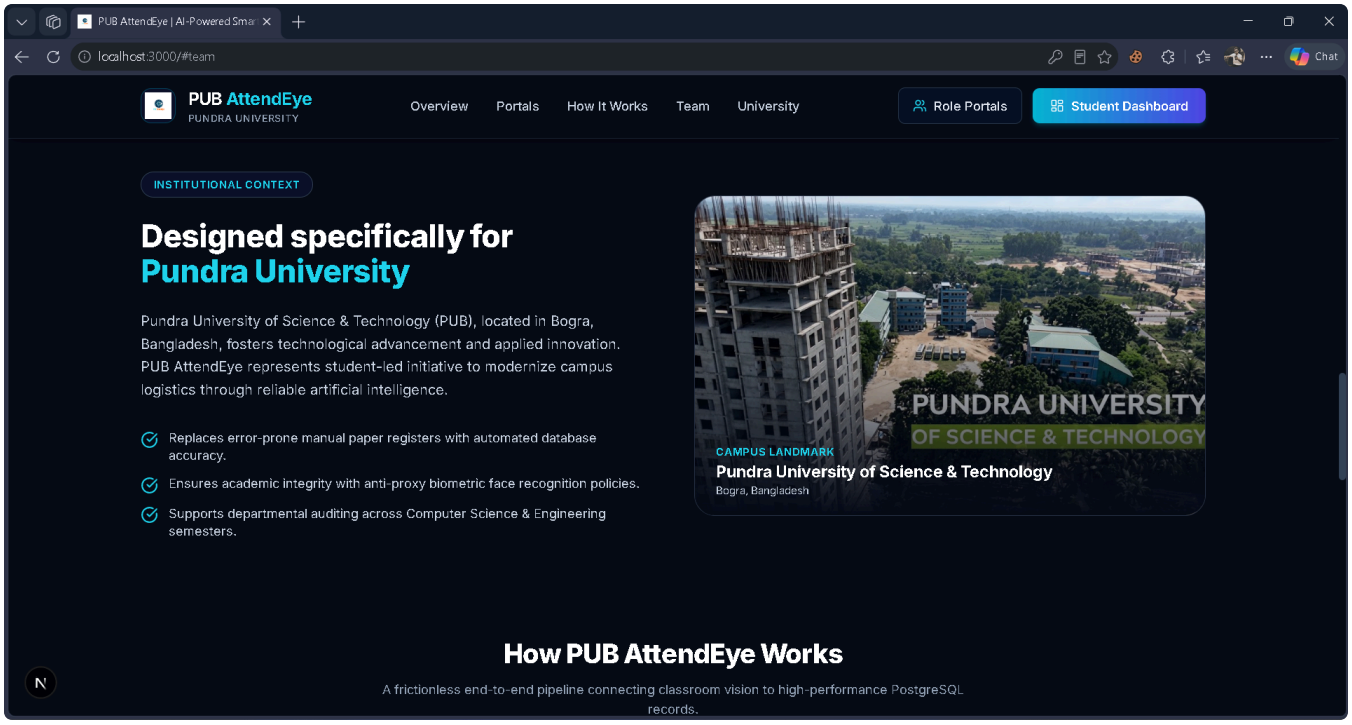


Figure 13. Institutional context section for Pundra University of Science & Technology, Bogra.